

Sistemas Distribuídos e Computação em Nuvem

Aula 11 — Consenso (Raft) e resiliência

C2 · Coordenação e consistência · 22/10/2026

Prof. M.Sc. Howard Cruz Roatti · FAESA · 2026/2

Onde estamos — C2

C1 · Fundamentos (1–7) ✓

C2 · Coordenação e consistência (8–12)

Mensageria ✓ · relógios ✓ · CAP ✓ · **Raft + resiliência.**

🧩 No CAP vimos que, sob partição, as réplicas **divergem**. Hoje a pergunta vira: como um grupo de réplicas **concorda em um único valor** apesar de falhas? E como **blindar** a parte mais frágil — a chamada de rede à IA.

C3 · Nuvem e segurança (13–18)

📌 **Aula 11.** Última aula de conteúdo novo da C2 — a Aula 12 é revisão + prova.

Retomada

 Você rodou o experimento da partição e escreveu o `relatorio_cap.md` com as duas justificativas (CP e AP)?

- CAP mostrou o **problema**: cópias que divergem.
- Hoje vem a **ferramenta** que muitos sistemas usam para o lado **C**: um **protocolo de consenso** que elege um **líder** e mantém um **log replicado** — o **Raft**.

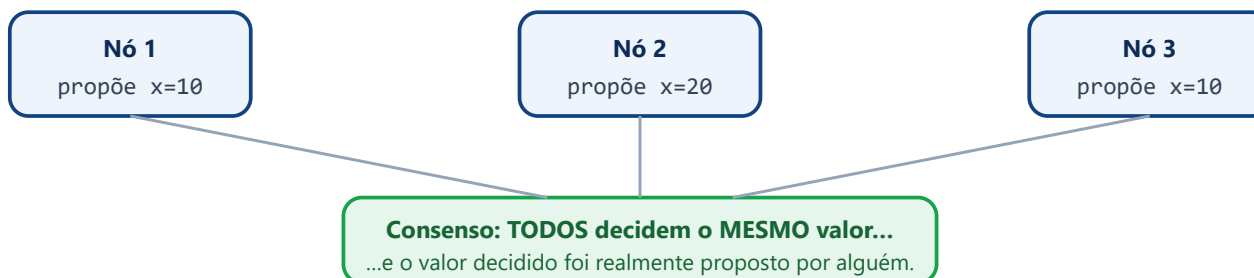
Objetivos desta aula

Ao final, você será capaz de:

1. **Explicar** o problema do **consenso** e o papel da **maioria (quórum)**.
2. **Descrever** o **Raft**: eleição de líder e log replicado.
3. **Blindar** uma chamada remota com **tempo-limite, retentativa e disjuntor**, apoiada em **idempotência**.

O problema do consenso

Várias réplicas, cada uma recebendo pedidos. Elas precisam **concordar em um único valor** (ou numa **ordem** única de operações) mesmo que **mensagens se percam** e **máquinas caiam**.



💡 **Em miúdos:** é um grupo decidindo onde almoçar por **voto**. Não precisa de unanimidade — basta a **maioria** fechar, e ninguém pode "trocar o voto" depois de combinado.

Raft em uma ideia: um líder manda

Em vez de todos discutirem entre si (difícil), o Raft **elege um líder**. O líder recebe as escritas, **ordena** e **replica** para os seguidores. Se o líder cair, os seguidores **elegem outro**.

- Simplifica: só o **líder** decide a ordem → todos seguem a **mesma** sequência.
- Tudo gira em torno de **mandatos** (*terms*): cada eleição abre um mandato numerado.

⚠ A peça-chave é a **maioria (quórum = $N/2 + 1$)**: eleger líder e confirmar uma escrita exigem maioria. Com 5 nós, a maioria é 3.

Raft · Eleição de líder (por maioria)



💡 Cada nó vota **uma vez por mandato**. Quem junta a **maioria** vira líder. Empatou/ninguém fez maioria? Abre-se um **novo mandato** e tenta de novo.

Raft · Log replicado (commit por maioria)

O líder anexa a entrada ao seu **log** e manda aos seguidores. A entrada é **comitada** (vale de verdade) quando a **maioria** gravou.

passo	o que acontece
1	líder recebe <code>x=10</code> , grava no próprio log
2	envia a entrada aos seguidores
3	maioria confirma a gravação
4	líder marca COMITADA e avisa o cliente

📌 Ligando ao **CAP**: exigir **maioria** é o preço da **consistência (C)**. Se a rede particiona e um lado fica **sem maioria**, esse lado **para** de aceitar escritas (abre mão de **A**) — Raft é um sistema **CP**.

Quando o líder cai

- Os seguidores param de receber o "sinal de vida" do líder → suspeitam da queda.
- Um deles vira **candidato**, abre um **mandato novo** (número maior) e pede votos.
- Com a **maioria**, assume e o sistema **continua** — as entradas já comitadas **permanecem**.

💡 É o que o `raft_sim.py` mostra: o Nó 1 cai, o Nó 2 abre o mandato 2, junta maioria e a vida segue — sem perder o `x=10` que já tinha sido comitado.

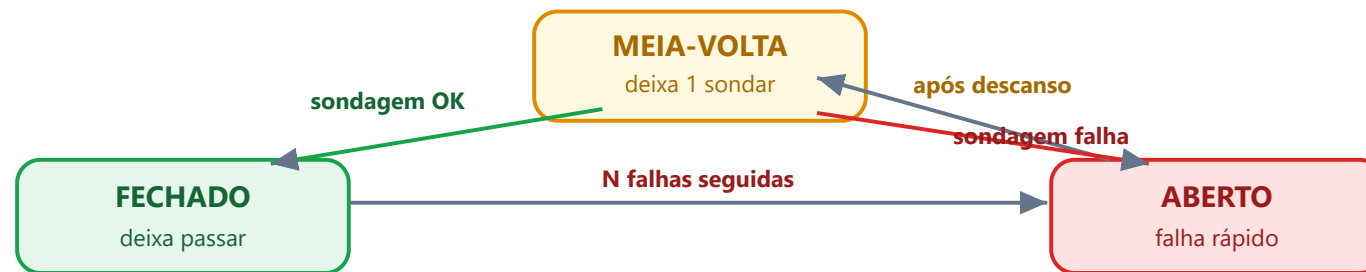
A outra metade: resiliência da chamada de IA

A chamada ao serviço de IA (ou a qualquer serviço remoto) é o ponto **mais frágil**: a rede atrasa, o serviço cai, a resposta se perde. Três defesas:

- **Tempo-limite (timeout)**: não espere para sempre — desista após X e trate como falha.
- **Retentativa (retry)**: tente de novo algumas vezes (idealmente com **espera crescente**).
- **Disjuntor (circuit breaker)**: se o serviço está claramente fora, **pare de insistir** por um tempo.

💡 Sem timeout, uma única chamada travada pode **prender** um worker para sempre. Sem disjuntor, mil retentativas **afundam** um serviço que já está mal.

O disjuntor (circuit breaker) — 3 estados



Fechado → (N falhas) → Aberto → (descanso) → Meia-volta → (OK) → Fechado

💡 **Em miúdos:** é o **disjuntor do quadro de luz**. Deu curto demais, ele desarma (abre) para não queimar a casa. Depois você religa com cuidado (meia-volta) e, se estiver tudo bem, deixa ligado (fechado).

Idempotência: a garantia que permite retentar

- **Retentar** é seguro **só** se repetir a operação **não duplica efeito**. Isso é **idempotência** (visto na Aula 5).
- `GET`, `PUT` e `DELETE` são idempotentes; um `POST` de criação **não** é.
- Truque comum: dar à requisição uma **chave de idempotência** (um id único) para o servidor **ignorar** uma repetição.

⚠ Retentativa **sem** idempotência = cobrar o cartão **duas vezes**. Antes de pôr retry, pergunte: "repetir isso faz mal?".

Laboratório

Ver o Raft acontecer e blindar a chamada de IA

Mão na massa — siga o Guia do Laboratório

Hoje são **4 missões**: ver a **eleição de líder** e o **log replicado** na simulação; **blindar** uma chamada com timeout + retentativa + disjuntor; e **concluir o C2.A2**.

👉 **Abrir o Guia do Laboratório »**

no índice do site: Aula 11 → [Guia do Lab](#)

💡 Aquecimentos prontos e testados: `exemplos/aula11/raft_sim.py` (eleição + log) e `exemplos/aula11/resiliencia.py` (timeout + retry + disjuntor).

O que as simulações mostram (aquecimento)

```
python raft_sim.py          # Nó 1 cai -> Nó 2 abre mandato 2 -> maioria -> segue  
# => 'x=10' COMITADA (maioria) ... depois ... No 2 eleito LIDER (mandato 2)
```

```
python resiliencia.py       # disjuntor em acao  
# [t2] FALHOU ... | disjuntor=aberto (falhas=3)  
# [t3] disjuntor ABERTO -> falha rapido (nem chama o servico)  
# [t4] descanso terminou -> MEIA-VOLTA -> OK -> disjuntor=fechado
```

💡 Os dois pilares da aula, rodando: **maioria** (consenso) e **falhar rápido** (resiliência).

No seu trabalho — C2.A2

- O **serviço de geração** do RAG chama um **LLM por API** — a parte mais sujeita a lentidão e falha.
- O **núcleo do trabalho** pede um **circuit breaker** nessa chamada: timeout + retentativa + disjuntor, exatamente o de hoje.

💡 Puxa para o kit: proteja a chamada ao LLM com o padrão do `resiliencia.py`. Garanta que a operação é **idempotente** (ou use uma chave) antes de retentar.

Atividade para casa

1. **Rode** `raft_sim.py` e `resiliencia.py` e **responda** ao roteiro do Guia (eleição, maioria, estados do disjuntor).
2. **Blinde** a chamada ao LLM no C2.A2 com **timeout + retentativa + disjuntor**.
3. **Conclua o C2.A2**: RAG distribuído (ingestão, recuperação, geração) **funcionando de ponta a ponta**.

📌 **Entregar até a próxima aula (que é a prova C2.A1)**: o RAG completo com a chamada ao LLM protegida. Roteiro completo no **Guia do Laboratório**.

◆ Foco ENADE

O que costuma cair:

- **Consenso** e **quórum/maioria**; eleição de líder e log replicado (**Raft**).
- Relação **Raft** ↔ **CAP** (exigir maioria = lado **CP**).
- **Resiliência**: timeout, retentativa (com backoff), **disjuntor** e seus estados.
- **Idempotência** como pré-condição da retentativa segura.

Termos-chave: Consenso · Quórum · Raft · Líder · Log replicado · Timeout · Retry · Circuit breaker · Idempotência

💡 Pegadinha: "pôr retry resolve". Só resolve se a operação for **idempotente** — senão, duplica efeito.

Questão de autoavaliação (estilo ENADE)

Em um sistema que usa **Raft** com **5 réplicas**, uma **partição** separa o grupo em **3 nós de um lado** e **2 do outro**. Sobre a aceitação de **novas escritas** durante a partição, é correto afirmar que:

- A) nenhum lado aceita escritas, pois Raft exige unanimidade
- B) ambos os lados aceitam, cada um com seu líder
- C) **apenas o lado com 3 nós aceita, pois só ele forma maioria (3 de 5)**
- D) apenas o lado com 2 nós, por ter menos conflito
- E) os dois lados param até um administrador intervir

Resolução — alternativa C

- Raft comita por **maioria** = $5/2 + 1 = 3$.
- O lado com **3 nós** forma maioria → **elege líder e aceita** escritas.
- O lado com **2 nós não** alcança maioria → fica **indisponível** para escrita (preserva **C**, abre mão de **A** → **CP**).

💡 É o elo Raft ↔ CAP: a maioria garante um único líder e evita divergência — ao custo de indisponibilizar a minoria.

Fora da sala · Glossário

Para estudar

- **Coulouris**, cap. 15 — Coordenação e acordo.
- *The Raft Paper* (Ongaro & Ousterhout) e a animação thesecretlivesofdata.com/raft.
- Rode `resiliencia.py` mudando o roteiro: provoque o disjuntor a reabrir.

Glossário

- **Consenso**: acordar um único valor apesar de falhas.
- **Quórum**: maioria necessária ($N/2 + 1$).
- **Raft**: consenso por líder + log replicado.
- **Timeout**: limite de espera de uma chamada.
- **Disjuntor**: corta chamadas a um serviço que falha muito.
- **Idempotência**: repetir não muda o efeito final.

Até a próxima aula

Conclua o C2.A2 (RAG completo, chamada ao LLM protegida) — a próxima é a **prova C2.A1**.

Próxima (Aula 12): Revisão e Avaliação C2.A1 — o caminho da C2 (mensageria → relógios → CAP → Raft) e as 6 perguntas que você precisa saber responder.

← Anterior
Aula 10 · CAP

≡ Índice
todas as aulas